Multivariate Time-Series Prediction using Deep Learning

Project Report

Candidate Name – Faara Ramsy

Date – 28th July 2026

Table of Contents

List of Figures

1. Introduction

This report documents the development of a multivariate time-series pipeline to predict household appliance energy consumption (Appliances, measured in Wh) using the UCI Appliance Energy Prediction dataset. The dataset consists of 19,735 records collected at 10-minute intervals over approximately 4.5 months (11 January – 27 May 2016), combining indoor sensor readings (temperature, humidity across 9 zones), outdoor weather data, and the energy consumption target.

The objective was not only to do preprocessing and build an accurate predictive model, but to demonstrate the handling of the time-series-specific challenges this task present; preventing data leakage, engineering features that respect the sequential nature of data, and evaluation of models fairly. The pipeline evaluates five baseline models (naive persistence, Decision Tree, Random Forest, XGBoost, Linear Regression) alongside four deep learning architectures (LSTM, GRU, CNN-LSTM, Bidirectional LSTM), with hyperparameter tuning applied to all four deep learning models via Optuna.

2. Data Insights

2.1 Dataset Overview

The Multi-Variate Appliance Energy Prediction dataset consist of 19,735 records across 28 features, recorded at 10-minute intervals from 11th of January 2016 to 27th of May 2016. A completeness check confirmed zero missing values and zero duplicate rows, so no imputation or any other handling was required for this stage.

2.2 Target Variable Distribution

The target variable, “Appliances (Wh)”, is strongly right skewed, most of the readings cluster between roughly 40-100 Wh, representing typical baseline household consumption, with a long thin trail extending past 900 Wh which might be because of infrequent high consumption events.

To minimize the effect of these events on our baseline models, log transformation was applied to the target variable as result of normalization.

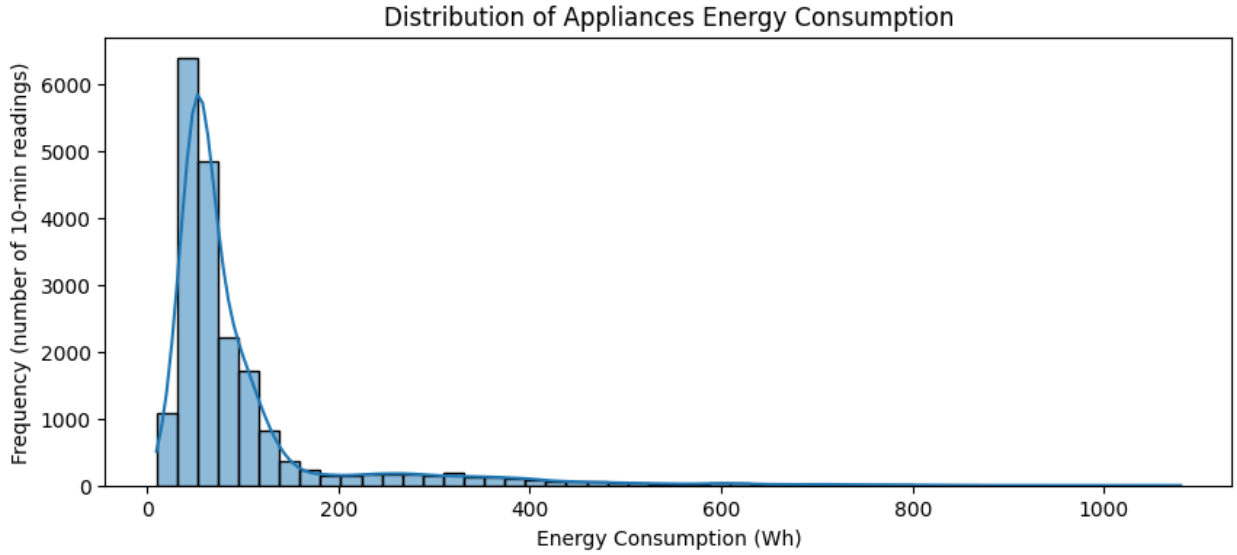


Figure 1:Distribution of the target variable, 'Appliances'

2.3 Temporal Patterns

Because Energy consumption is a time-dependent process, temporal analysis was the primary focus of EDA.

- Hour of the day trend: Average consumption by hour as shown in Figure 2 shows a clear daily cycle, minimum usage overnight, and a steady rise from 7.00, and a peak between 17.00 and 19.00, which is because of the users returning home and using the appliances

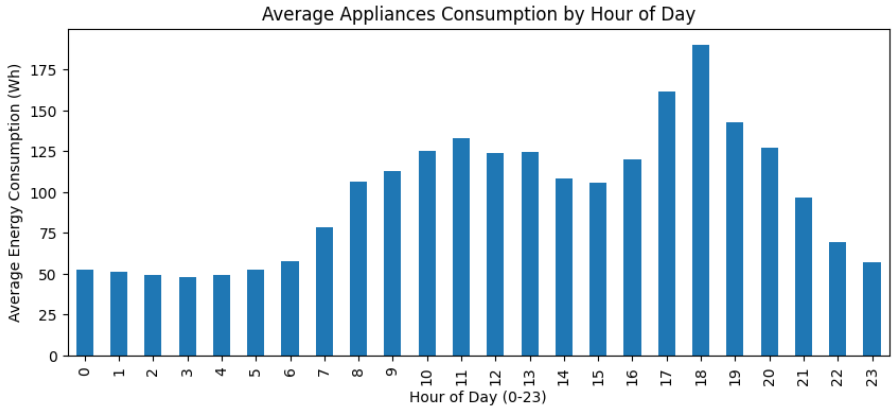


Figure 2:Hour of the day trend

- Day of the week trend: Moderate variation does exist (Monday and Tuesday trend is higher than Tuesday/Wednesday) even though the effect is smaller than the hourly patterns on its own

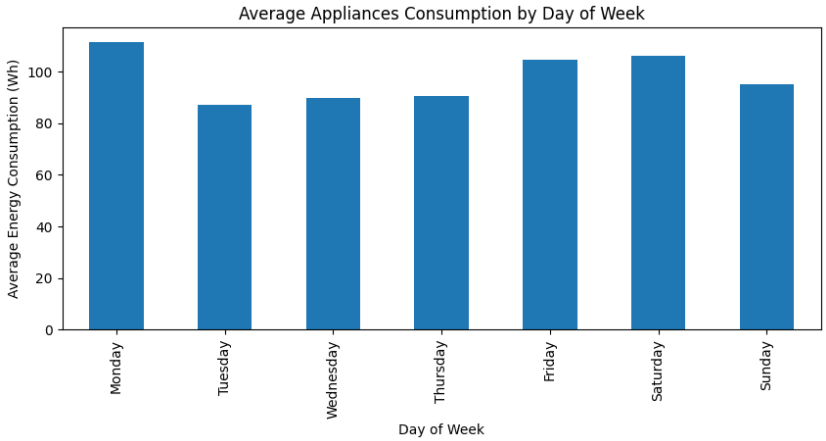


Figure 3: Day of the week trend

- Hour & Day of the week interaction heatmap: Reveals what neither of the charts show alone, weekend evenings are sharp and concentrated (17.00 -19.00) as the occupants return home, while weekend peaks are broader and shifter later due to delayed weekend routine.

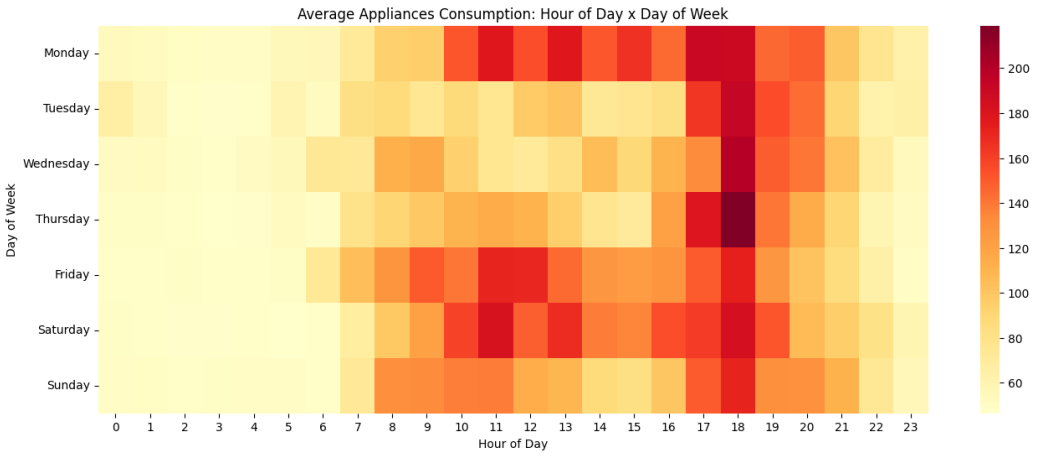
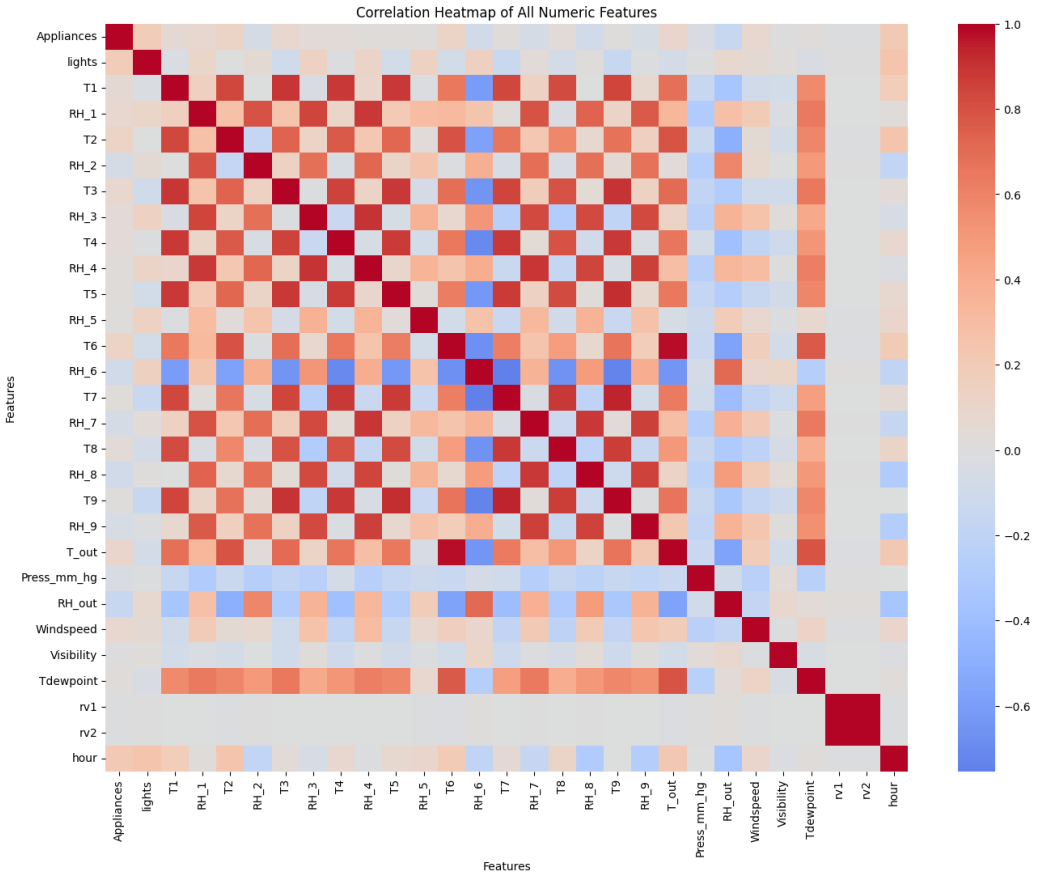


Figure 4: Hour and Day heatmap

2.4 Correlation Structure

A Correlation matrix across all numerical features showed that no single feature strongly correlated linearly with the target variable ‘Appliances’. The strong positive was the hour and lights with 0.20, the strong negative was RH_out with -0.15.

The absence of a strong single linear predictor indicates Appliances is not well explained by raw environmental readings alone. This is a factor which motivates the engineering and interaction features developed in Section 4.

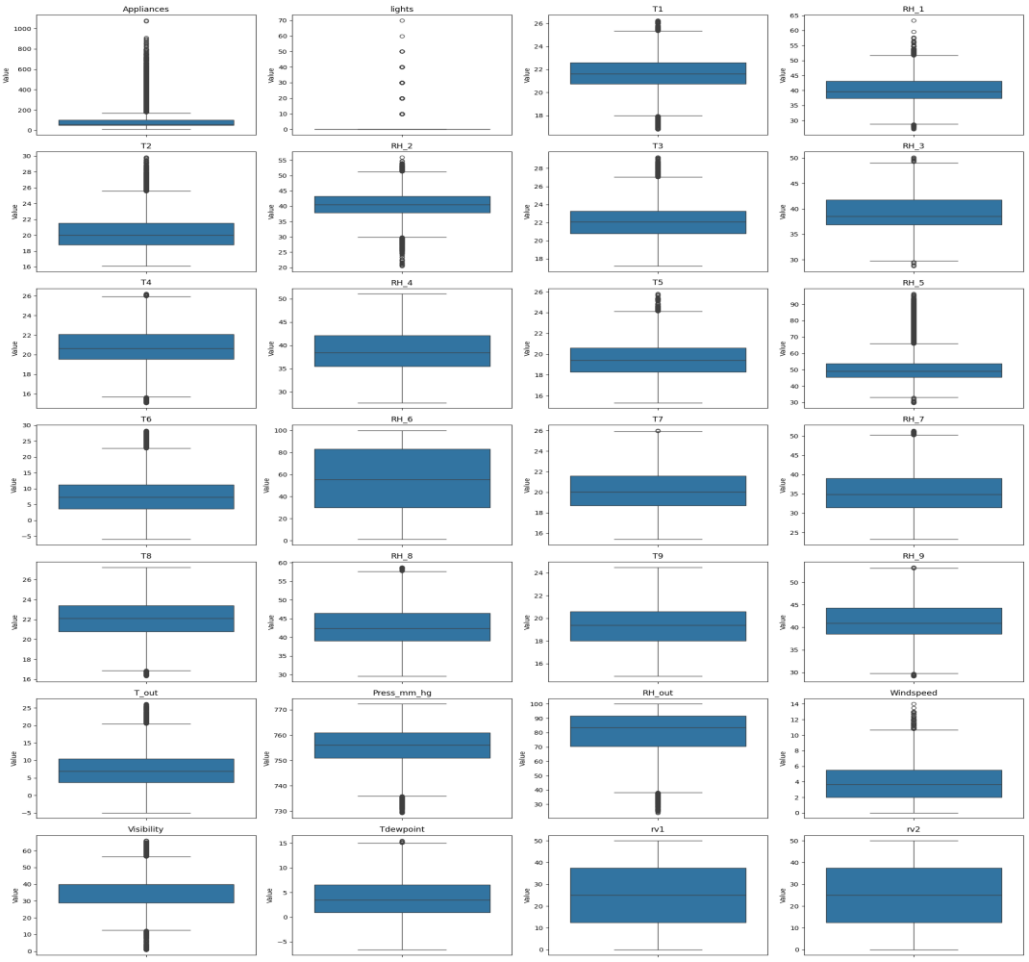



```
...  Appliances     1.000000
     hour           0.216792
     lights         0.197278
     T2             0.120073
     T6             0.117638
     T_out          0.099155
     Windspeed      0.087122
     RH_1           0.086031
     T3             0.085060
     T1             0.055447
     T4             0.040281
     T8             0.039572
     RH_3           0.036292
     T7             0.025801
     T5             0.019760
     RH_4           0.016965
     Tdewpoint      0.015357
     T9             0.010010
     RH_5           0.006955
     Visibility     0.000230
     rv1           -0.011145
     rv2           -0.011145
     Press_mm_hg   -0.034885
     RH_9          -0.051462
     RH_7          -0.055642
     RH_2          -0.060465
     RH_6          -0.083178
     RH_8          -0.094039
     RH_out        -0.152282
     Name: Appliances, dtype: float64
```


Figure 5: Correlation heatmap and values of all the numerical features

2.5 Outlier Landscape

Boxplots and IQR-based outlier count showed outlier percentage varying widely: Lights with 22.7%, Visibility 12.8%, Appliances 10.8% and RH_5 6.7% had the highest flagged proportions. A target check on RH_5 showed that these flagged points reflected genuine environmental variation, not sensor faults.



	count	pct
lights	4483.0	22.72
Visibility	2522.0	12.78
Appliances	2138.0	10.83
RH_5	1330.0	6.74
T2	546.0	2.77
T6	515.0	2.61
T1	515.0	2.61
T_out	440.0	2.23
T_indoor_avg	341.0	1.73
RH_out	239.0	1.21
RH_2	235.0	1.19
temp_diff	219.0	1.11
Press_mm_hg	219.0	1.11
T3	217.0	1.10
Windspeed	214.0	1.08
T4	186.0	0.94
T5	179.0	0.91
RH_1	146.0	0.74
T8	71.0	0.36
RH_7	42.0	0.21
RH_9	21.0	0.11
RH_8	17.0	0.09
RH_3	15.0	0.08
Tdewpoint	11.0	0.06
T7	2.0	0.01
RH_6	0.0	0.00
RH_4	0.0	0.00
T9	0.0	0.00
rv1	0.0	0.00
rv2	0.0	0.00
RH_indoor_avg	0.0	0.00

Figure 6:Outlier Percentage across all the features

2.6 Autocorrelation

ACF/PACF plots of Appliances showed significant short lag autocorrelation, confirming recent past values carry strong predictive information about the near future, This is the direct justification for the lagged and rolling features engineering in Section 4, for using temporal train /test split – later validated numerically, as Appliances_lag_1 was to have found about 37% of Random Forest feature importance.

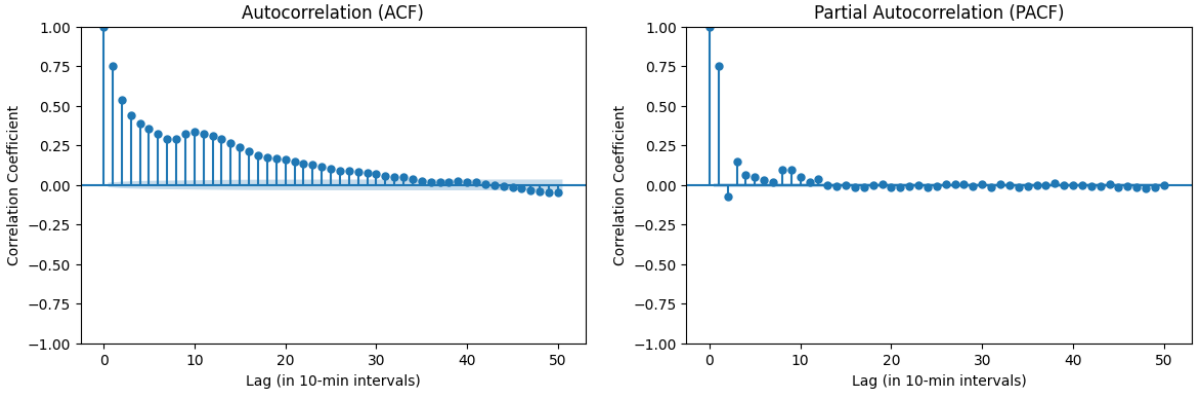


Figure 7ACT/PACF plots of the target variable, Appliances

2.7 Physical Plausibility checks

Scatter plots of indoor temperature vs indoor humidity (r = -0.42) and outdoor temperature vs. outdoor humidity (r = -0.57) both show the expected inverse relationship dictated by basic thermodynamics. This consistency with known physical behavior increases confidence in overall data quality.

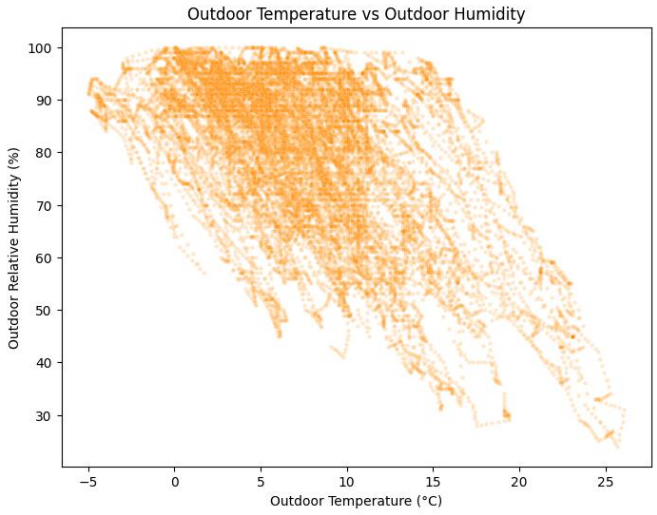


Figure 8: Outdoor relative humidity vs Outdoor temp

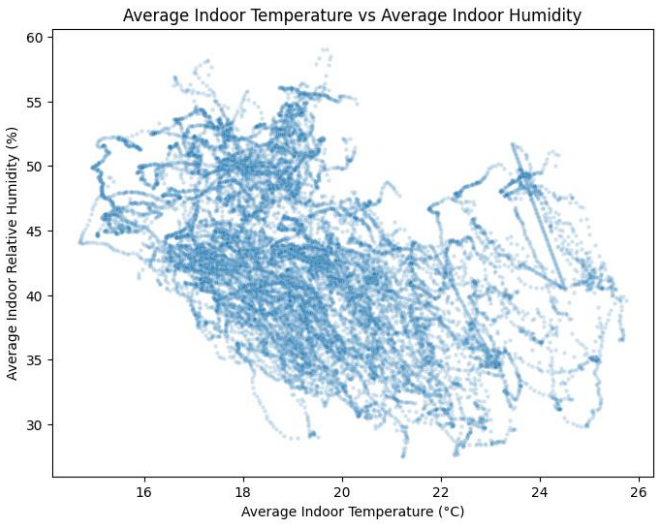


Figure 9:Average Indoor Humidity vs Average Indoor temperature

3. Preprocessing

3.1 Missing Value Handling and Duplicate Records

df.isnull().sum() confirmed zero missing values across all 28 columns, therefore no imputations were required

df.duplicated().sum() confirmed zero duplicate rows; there was no handling required.

3.2 Outlier Detection and Treatment

Outliers were identified using the IQR method (values outside Q1 − 1.5×IQR to Q3 + 1.5×IQR), cross-validated visually with boxplots.

Outliers were retained rather than removed, for two reasons:

- High-outlier features (Appliances, lights, Visibility) are naturally right-skewed, legitimate and removing them (especially in the target variable itself) would discard real, meaningful signal.
- A spot-check on RH_5 confirmed its extreme values are physically valid readings, not sensor errors.

Instead of removing outliers, the target variables skewness was addressed by doing log-transformation which acted as an alternation to the raw target, especially for the Linear Regression baseline (See Section 6.1 for the comparative result)

3.3 Train/Test Split

The dataset was split chronologically – the first 80% of the time for training, the final 20% for testing but with no shuffling, to prevent look ahead leakage inherent to time series data, Train: 15,672 rows and Test: 3919 rows

3.4 Scaling

Scaling was applied per model family, based on each model's sensitivity to feature magnitude:

- Linear Regression – No scaling applied, OLS is scale-invariant, so it used the same unscaled features as the tree-based models
- LSTM/ GRU/ CNN-LSTM: MinMaxScaler. To bound inputs to a stable for gradient descent convergence.
- Random Forest / XG Boost: No scaling applied, as it's a tree-based models and they split on threshold rather than distances

In all cases, scalers were fit exclusively on the training dataset and then applied to the test set, to prevent test statistics from leaking into training.

4. Feature Engineering

The purpose of this stage was to transform 28 raw columns of the dataset into a feature set that the models could realistically learn, as no individual sensor reading strongly correlated with the target (most was hour with r =0.2).

4.1 Time based features

Hour, day of week, and month were extracted and encoding using cyclical (sine/cosine) transformation rather than raw integers, so that adjacent time boundaries (ex: 0 and 23 hour) are represented numerically closer, matching their truth temporal adjacency.

A weekday/ weekend binary indicator was also added.

4.2 Rolling Window Statistics

Rolling mean and rolling standard deviation were computed over 1 hour and 3 hour windows. Standard deviation as included alongside the mean because rising short-term variance can show an energy-usage spike, a type of information mean cannot capture alone.

4.3 Lagged Features

Lag features were created at 1,2,3,6,18, and 144 steps (10 minutes through 24 hours). The first three lags were selected directly because of the PACF analysis in Section 2.7, the remaining lags such as 1 hour, 3hr and 24 hours were added based on domain reasoning, HVAC response time, daily seasonality, respectively.

4.4 Interaction Features

An indoor–outdoor temperature differential (T_indoor_avg − T_out) was engineered to capture the driver of heating/cooling-related energy use more directly than either temperature alone. Explicit temperature * humidity interaction terms were also created for both indoor and outdoor readings, motivated by the non-linear structure observed in the heatmap.

4.5 Domain-Specific Features

A lights_on binary flag was derived from the continuous light's variable, as a simpler representation for activity. A public holiday indicator (is_holiday) was also engineered against the observation window, as it would also influence the appliance usage. It was therefore considered an external factor

4.6 Feature Selection

Feature Selection was deliberately performed after the Feature Engineering, using two independent methods on the training set only:

- Random Forest importance - Measures how much each feature contributed, on average.

- Recursive Feature Elimination (RFE). – Repeatedly trains a model and removes the weakest contribution feature and retrains until a specific number of features remain.

Random Forest importance showed Appliances_lag_1 showed a feature importance of 37%, confirming PACF finding and establishing that short term persistence is the dominant signal in this dataset.

The final feature set (features selected by both RF importance and RFE, with hour_sin/hour_cos retained despite narrowly missing the strict intersection, given their strong independent correlation evidence from EDA) comprised: Appliances_lag_1, Appliances_lag_2, Appliances_lag_144, Appliances_roll_mean_1hr, Appliances_roll_mean_3hr, Appliances_roll_std_1hr, Appliances_roll_std_3hr, Press_mm_hg, RH_1, RH_3, T3, T4, hour_sin, hour_cos.

The synthetic noise variables rv1 and rv2 were confirmed as non-informative by both methods (close-bottom importance ranking in both RF and RFE and with their −0.011 correlation from EDA) and excluded from the final feature set.

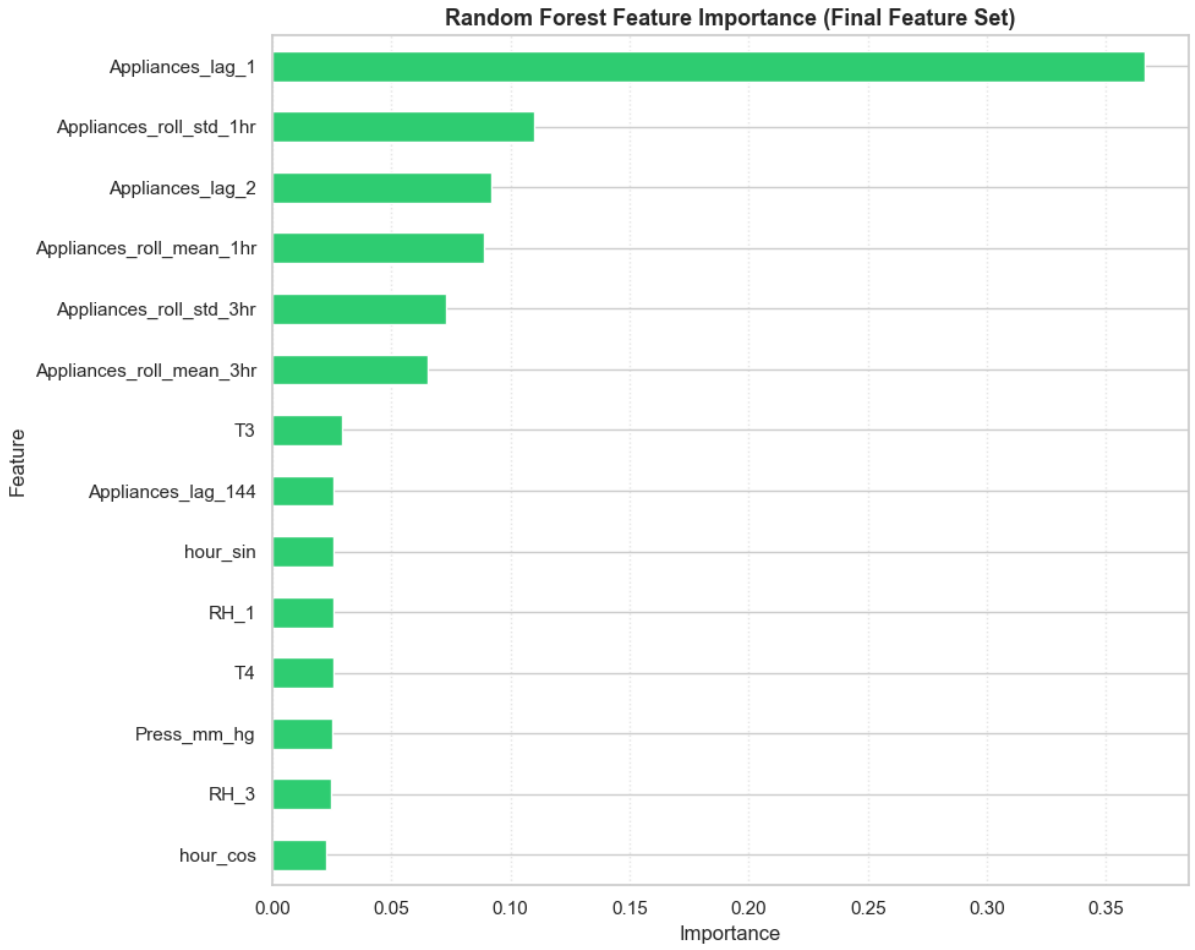


Figure 10:Random Forest feature importance

5. Model Design

Many models, increasing in complexity, were built and evaluated under identical conditions, the same finalized feature set, and the same chronological train/test spit and evaluation metrics. The base models used are a naïve persistence baseline, Linear Regression, a single feature Decision Tree, Random Forest, XGBoost and the four deep learning architectures were LSTM, GRU, CNN-LSTTM, and Bidirectional LSTM

5.1 Deep Learning Architectures

In order to isolate the impact of the recurrent mechanism itself from unrelated variations in network size, all four recurrent architectures were purposefully constructed on a common structural template: two stacked recurrent layers of 64 units each, followed by a Dense layer of 32 units, BatchNormalization, Dropout, and a single-unit linear output layer.

LSTM (Long Short – Term Memory) – This architecture was selected as the foundation architecture because is addresses a common limitation of simple Recurrent networks, the vanishing gradient problem where information from the early time steps in a sequence becomes harder to store for the network as the sequence gets longer.

This problem is solved by LSTM by using an internal cell state and three gating mechanisms: an input gate, a forget gate, and an output gate, that controls what data is kept, deleted, or passed along at each time step. Because the model must learn temporal dependencies over a 12-timestep (2-hour) input window, it is well suited to a sequence-based prediction problem like this one.

GRU (Gated Recurrent Unit) – This architecture was included as a lower capacity alternative to LSTM. Instead of three gates like the LSTM, there are only two gates (an update gate and reset gate) and removes the separate cell state entirely, only replying on a single hidden state. This results in meaningfully fewer trainable parameters than an equivalent LSTM layer. GRU was considered because of the results from the baseline models obtained earlier, which showed that less complexity models consistently outperformed higher complexity alternatives on this dataset.

CNN – LSTM – This architecture was used to capture both localized temporal dynamics and broader sequence dependencies, A 1D Convolution Neural Network layer acts as a feature extractor at the input stage, scanning across localized time windows to extract high level feature representations.

The output of the convolutional layer is then passed to the stacked LSTM layers which reason over the longer-range structure of the locally summarized sequence. Theoretically, the convolutional component acts as a local feature extractor, while the recurrent component performs longer-range temporal reasoning on top of it.

Bidirectional LSTM – This architecture processes each iput sequence in both the forwards and backward direct simultaneously, using two sperate LSTM passes whose outputs are

combines. This allows the network to draw on context from both the ends each time window forming its representation, compared to only past to present in a unidirectional LSTM.

5.2 Activation Functions

ReLu (Rectified Linear Unit) was used in the Dense hidden layer following the recurrent layers. This activation function was selected because it helps to avoid vanishing gradient problems in feed-forward layers.

Linear Activation was used on the final output layer, which is the standard and correct choice for regression tasks.

5.3 Loss Function

Huber Loss was selection over MSE (Mean Squared Error) for all four deep learning models. This choice was made because of the findings from the EDA, where the target variable is strongly right skewed, with most of the readings being concentrated at low values and a long tail of infrequent high- consumption spikes present.

While MSE penalizes large errors, it renders gradient based optimization which is very sensitive to extreme outliers. This forces the model to over-adjust temporary spikes, degrading overall performance on steady baseline demand. But combines the advantages of both MSE and MAE (Mean Absolute Error). It acts like MSE for small errors (ensuring smooth gradient updates near convergence) and transitions to MAE for large errors (preventing isolated spikes from disproportionately dominating the gradient step).

Even though these outliers were addressed by doing log transformation for the target variable, this reduced the performance for Linear Regression, therefore Huber Loss addressed the same skew problem without requiring the target variable itself to go through transformation

5.4 Optimizer

Adam was used to train all four deep learning models. Adam was selected because it combines the properties, momentum which smooths out noise, inconsistent gradient updates from one training batch to the other, and per -parameter adaptive learning rates, which also gets automatically adjusted based on that parameter's gradient history.

This is required because the finalized feature sets consist of several very differently scaled and differently behaved inputs. Adam's adaptive behavior reduces the need for extensive manual learning rate tuning across such a feature set and is also considered as a standard for this type of model

5.5 Regularization and Overfitting Prevention

The four techniques used to manage model complexity and control overfitting were:

- Dropout – Randomly deactivating a proportion of neurons during each raining step.
- Batch Normalization – Stabilized and standardizes the distribution of activation passed between layers, general improving training stability and convergence speed

- EarlyStopping- Monitors the validation set during training and stops the process once the performance stops improving and restoring the best performing set of weights.
- ReduceLROnPlateau – when validation loss stops improving for a set number of epoch, the learning rate automatically gets halved allowing the optimizer to take smaller steps.

6. Results

6.1 Baseline Model Comparison

model	MAE	RMSE	MAPE (%)	R2
Naive Persistence (t-1)	26.03	65.18	21.86	0.45
Decision Tree (lag_1 only)	28.46	61.36	26.93	0.51
Random Forest	41.98	66.48	46.69	0.43
XGBoost	55.77	79.84	68.93	0.17
Linear Regression (log target)	27.69	71.92	22.61	0.33
Linear Regression (standard)	26.84	58.98	25.18	0.549

Figure 11: Baseline model comparison

Four metrics were used to evaluate every model consistently.

- Mean Absolute Error (MAE) – The average of the absolute differences between predicted and actual values.
- Root Mean Squared Error (RMSE) – The square root of the average of squared differences between predicted and actual values.
- Mean Absolute Percentage Error (MAPE) – Average error as a percentage of the true value, providing a scale independent measure of accuracy.
- R2 – Explains how well a regression model explains the variance or variability in the target variable.

Linear regression trained on raw target was the strongest performer amount all baseline models with a R2 value of 0.549.

Log vs Raw Target Comparison

Given the right skewed distribution of the target variable identified through EDA, Linear regression as training on both raw and log transformed data. The raw target version performed better (as shown in the matrices above), indicating that the relationship between the engineered features and the target did not become more linear under log transformation in this case. Therefore, this transformation was not carried out with the deep learning models, Huber Loss was used instead to address the skewness issue.

6.2 Deep Learning Model Comparison

Before Optimization

model	MAE	RMSE	MAPE	R2
LSTM	37.13	72.267	37.801	0.323
GRU	35.32	73.12	34.65	0.3073
CNN-LSTM_v2	39.528	76.36	38.81	0.245
Bi-LSTM	39.037	73.005	41.22	0.3095

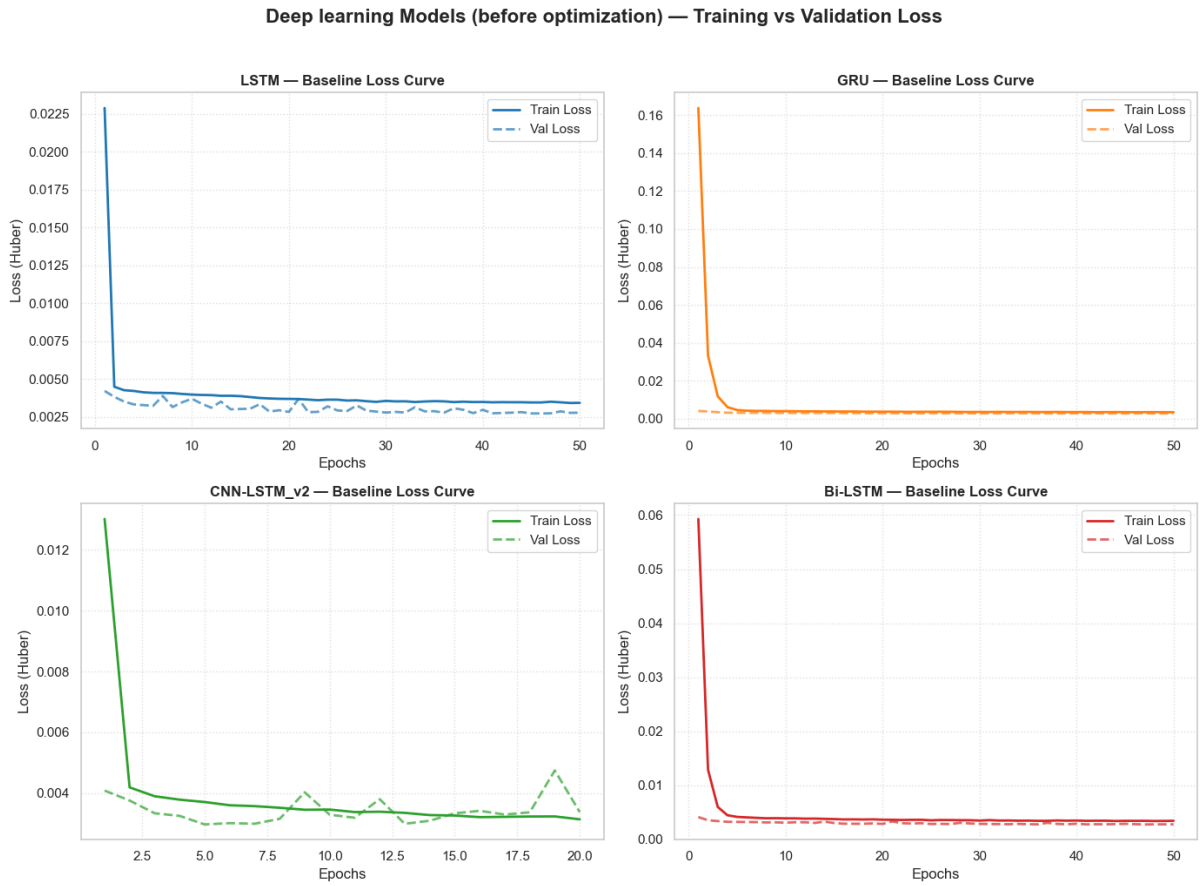


Figure 12: Training vs Validation loss of DL models before optimization

All four architectures were trained under identical conditions, the same finalized feature set and the same sequence window, the same loss function and optimizer. The training/validation loss curves show sharp initial drop from the first two to three epoches followed by stable plateau, with validation loss tracking which was below. This confirms that none of the four dep learning models were overfitting.

7. Model Optimization

Technique

Hyperparameter tuning was carried out using Optuna, applying its default sampling algorithm, the Tree-structured Parzan Estimator (TPE), with each architecture undergoing 10 independent trials. The hyper parameters searches were the number of units in each of the two recurrent layers (32–96 and 16–64), the dropout rate (0.1–0.4), the learning rate (1e-4 to 5e-3), and the batch size (32, 64, or 128).

1. Grid Search evaluates every combination within a defined range repeatedly, including combination in region already shown to perform poorly wasting computation effort.

2. Random Search samples combination without any memory of previous results, meaning it cannot lean to focus its search.

TPE, builds a probabilistic model mapping hyperparameter configuration to expected validation metrics based on historical trial outcomes. This approach avoids unnecessary full network training runs and reduces computation effort.

Each of the four architecture was tuned independently, with each trial trained on a dedicated validation split held out separately from both the training data used for final model fitting and test data used for final evaluation. Once the best performing combination was identified for a given architecture, it was used to train a final model to convergence using the same early stopping criteria as the models before optimizations. Then it was evaluated on the untouched test set.

```
================================================================================
FINAL SUMMARY: BEFORE VS. AFTER HYPERPARAMETER OPTIMIZATION
================================================================================
```

	Model Architecture	MAE (Base)	RMSE (Base)	R2 (Base)	MAE (Opt)	RMSE (Opt)	R2 (Opt)
0	Bi-LSTM	39.0370	73.0050	0.3095	34.4048	71.4834	0.3380
1	LSTM	37.1321	72.2567	0.3236	33.5074	71.8547	0.3311
2	GRU	35.3200	73.1208	0.3073	40.7265	72.3476	0.3219
3	CNN-LSTM_v2	39.5282	76.3579	0.2446	39.3555	72.4963	0.3191

Figure 13:Result before vs after optimization

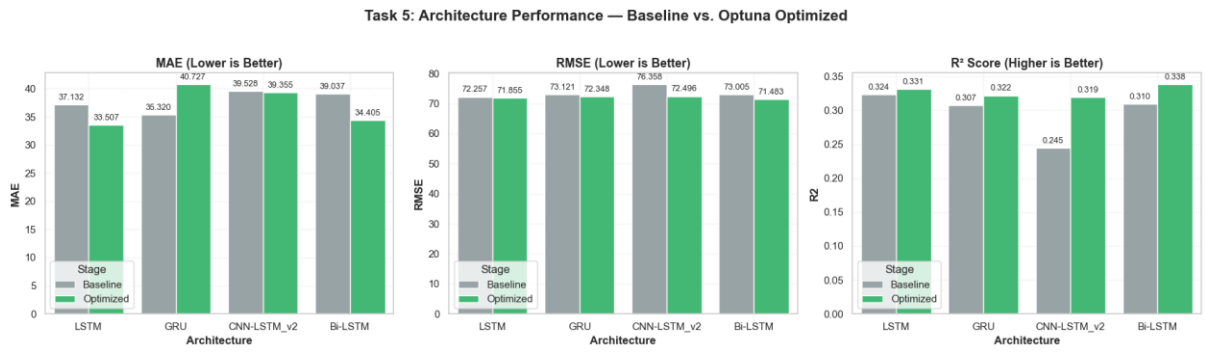


Figure 14: Performance barchart before and after optimization

Here Baseline refers to the deep learning models before optimization for comparison

Optimization improved the R2 for all the four architectures, though the difference differed between the models,

LSTM and Bidirectional LSTM improved consistently across every metric. These two models are the clearest wins from the fine-tuning process, with Bidirectional LSTM remaining the strongest deep learning architecture both before and after tuning.

CNN-LSTM showed a large improvement in R2 value and RMSE values, while the MAE remained almost unchanged.

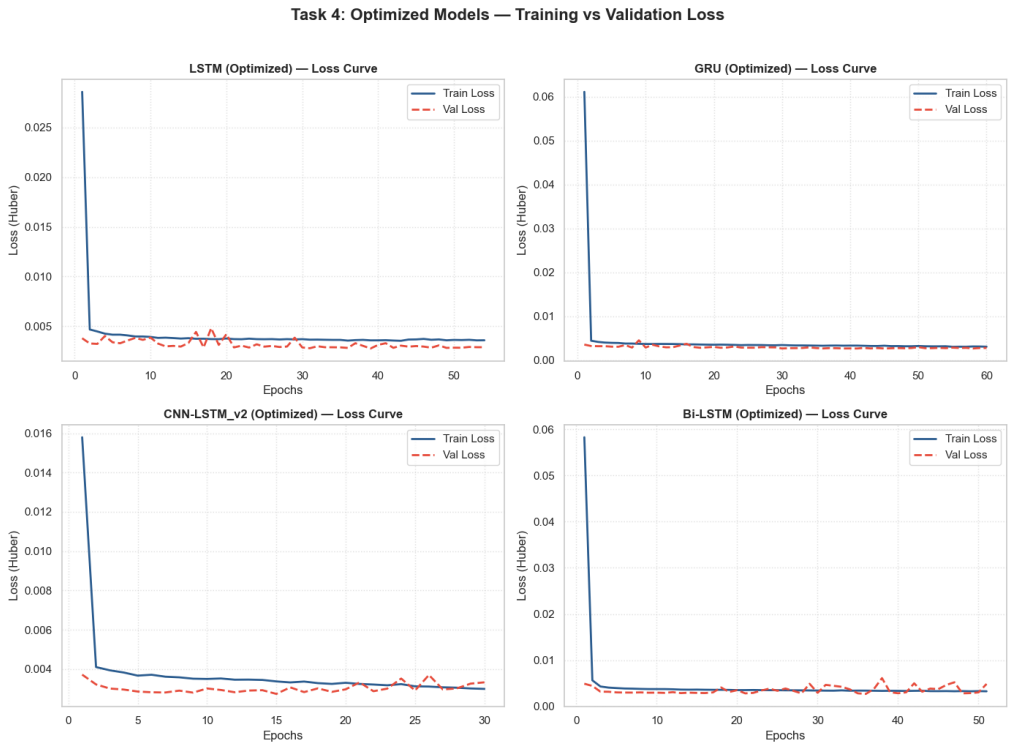


Figure 15: Validation/Training Loss curve

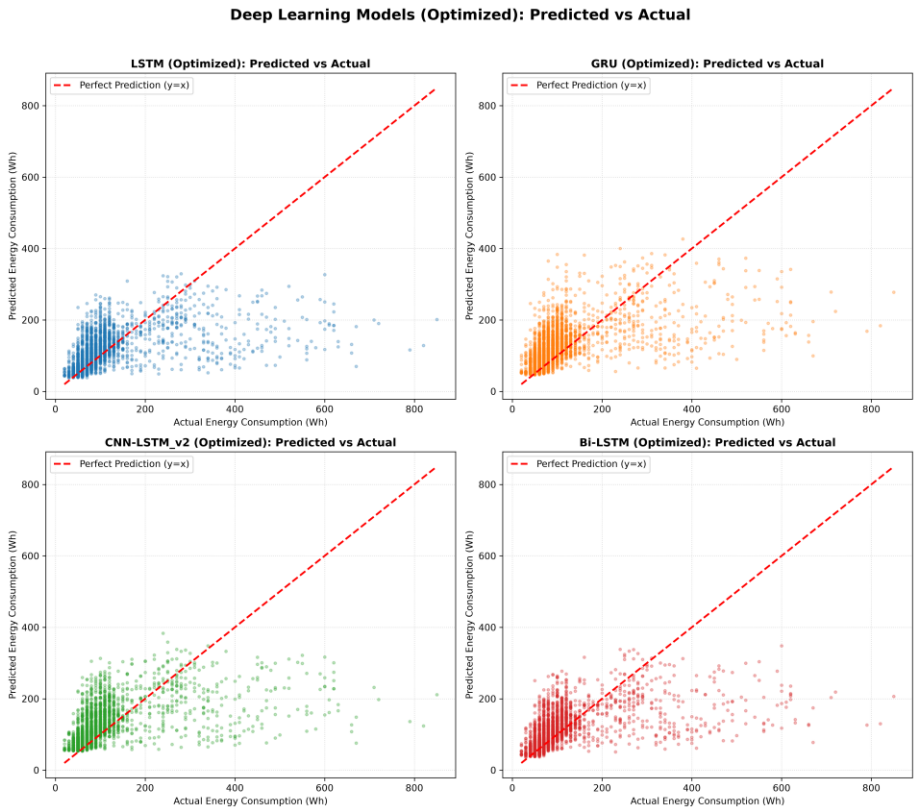


Figure 16: Optimized DL models: Predicted vs Actual

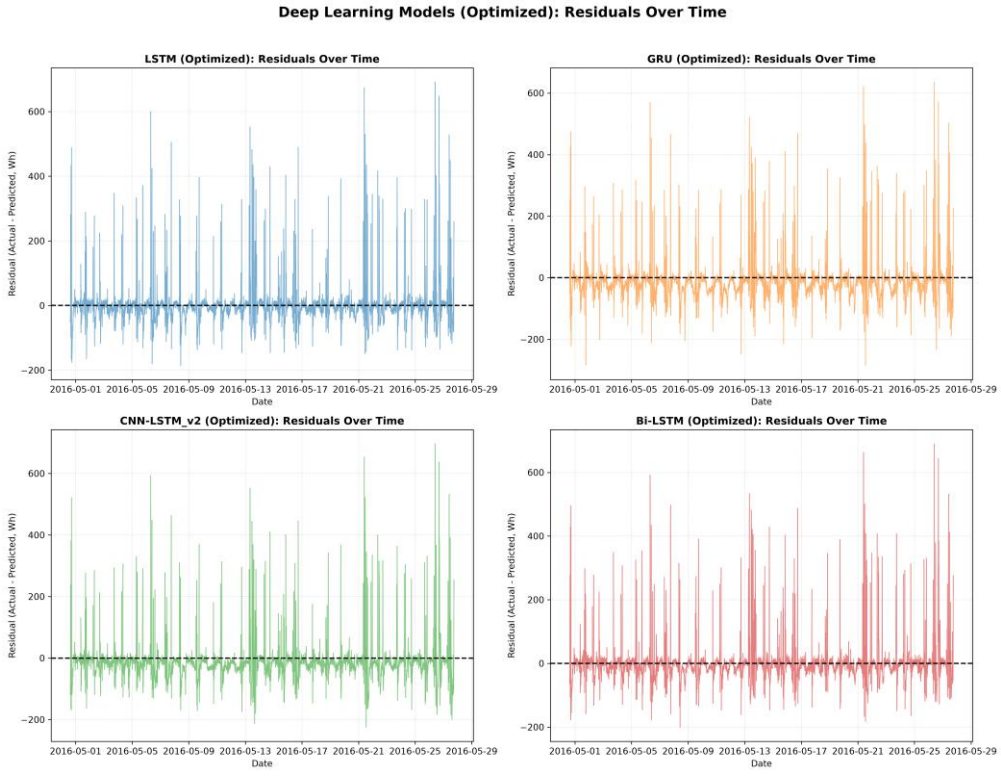


Figure 17: Optimized DL models (Residuals over time)

Why do the simple models have higher performance than complex deep learning architecture?

Across all the models, none of the models closed the performance gap with Linear regression baseline with a R2 value of 0.549. The strongest deep learning model Bidirectional LSTM, R2 value of 0.338 remained below it. This is the central analysis of this report, almost all the useful information for predicting current energy usage comes from just a few lag features, the dataset isn't large or complex enough to justify deep, heavy neural networks.

Tuning only helps a deep learning model to make better use of the information available; However, it cannot invent new predictive signals that don't exist in the data. If the Underlying data favors simple linear relationships, hyper parameters would not be able to turn a neural network into the top performer.

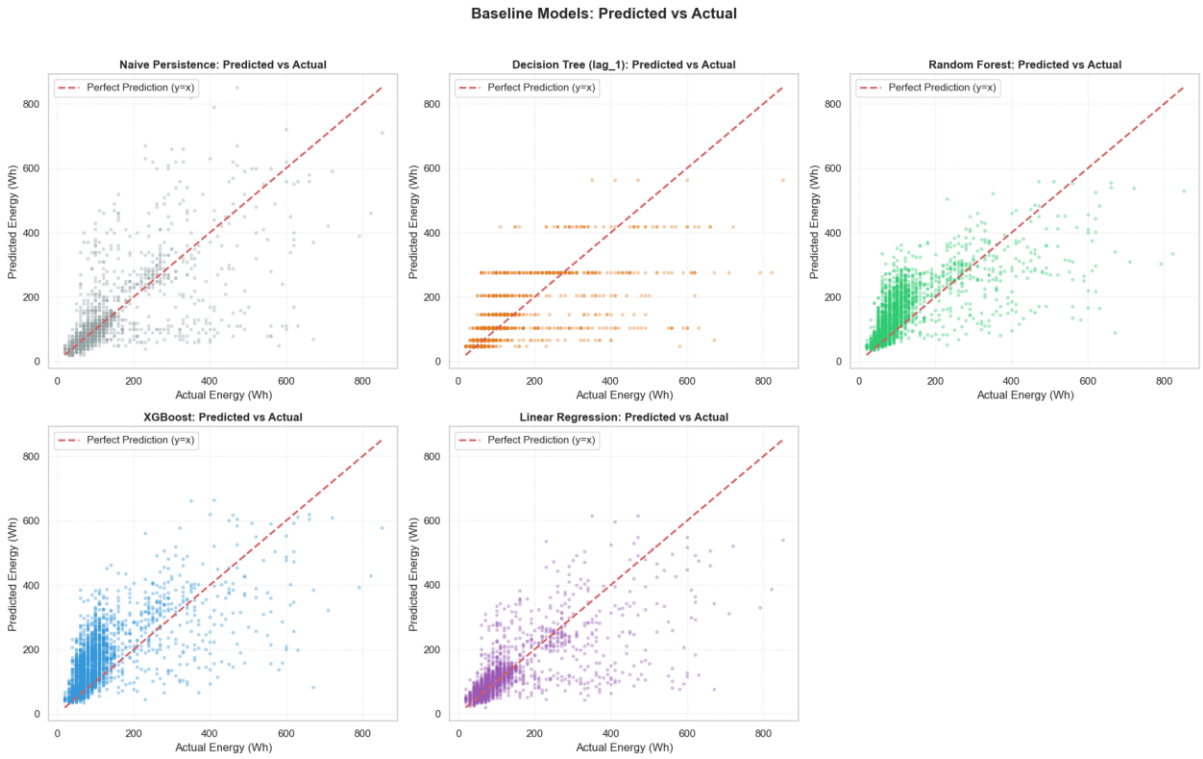


Figure 18: Baseline Predicted vs Actual plot

This scatter plot for the five baseline models shows the expected clustering along the diagonal reference line at low to moderate energy values. With increasing scatter at high energy consumption this demonstrated that models struggle to accurately predict rare, sharp consumption spikes due to heavy appliance usage.

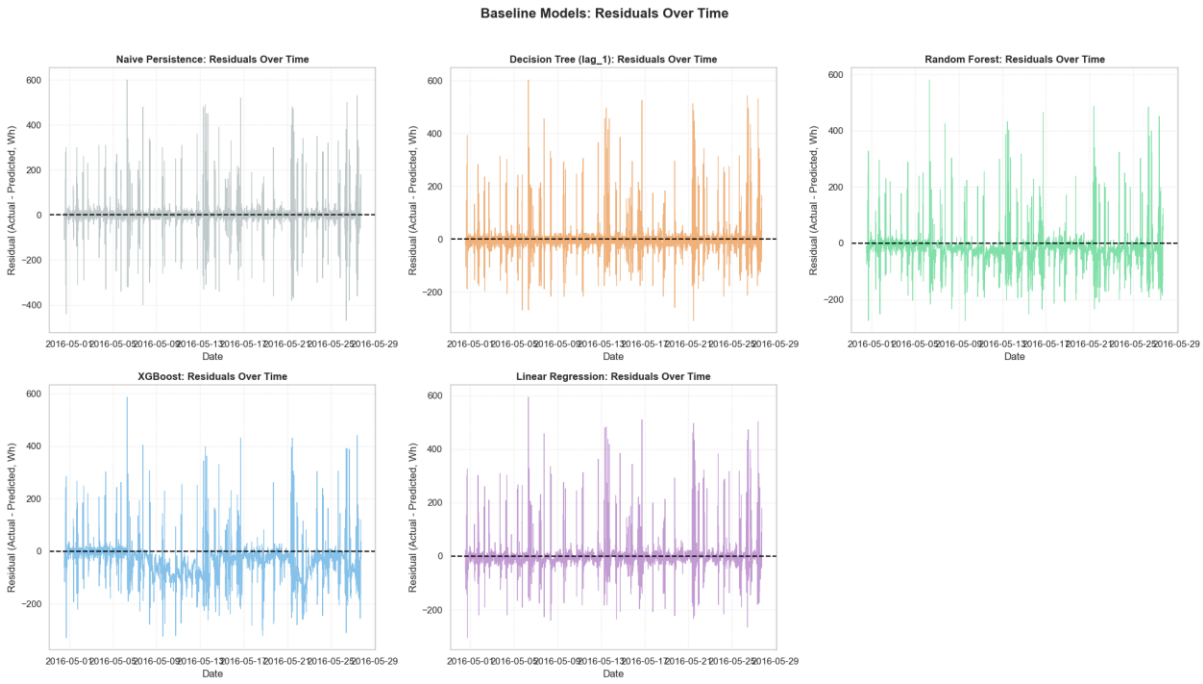


Figure 19: Baseline Models: Residuals over time

This shows that for models like Linear regression, residuals remain balances around zero indicating minimal systematic error. For tree ensembles like Random Forest and XG boost, these plots reveal upward skew which proves these models systematically over predict normal demand because they over adjusted to occasional high consumption spikes in the training dataset.

8. Challenges and Solutions

- Tree ensemble underperformance – Initial Random Forest and XG boost ran worse than Naïve baseline including outputting negative r2 values. Diagnosis showed this was driven by overfitting from unconstrained tree dept, adding max depth, min samples leaf and regularization parameters sorted the issue.
- CPU training tie constraints - Given the number of models and hyper parameter tuning trials required, CPU only training was estimated to take several hours.
- Data Leakage Issue in the feature selection pipeline - Early in the project, feature selection was run before the training and testing data had actually been separated

9. Conclusion

Across all the models with the baseline models and the Deep Learning architectures to do this project, the strongest overall performer on this dataset was Linear regression trained on raw target (R2 value =0.549). The best deep learning model, Bidirectional LSTM, reached a R2 value of 0.338 after optimization. This outcome is well supported by the evidence gathered throughout this project rather than being an unexplained weak result.

- Single Dominant feature – Over 37% of the predictive signal comes from just one feature, the first 10-minute lag for the target variable itself. All lagging and rolling historical features combined account for about 80% of total predictive weight.
- Complex Model fit to Noise – Because the primary driver is simple short-term persistence, Linear Regression used this single strong linear signal with no interruption. High-capacity models have too much parameter space, they might be overfit by trying to find complex relationships in weak features, rather than sticking to the primary driver

These findings are consistent with the original research behind this dataset (Candanedo et al., 2017), which similarly reported a Random Forest outperforming a neural network on the same data suggesting this is a genuine property of the dataset rather than an flow of this pipeline

Future works.

Automated Feature Extraction – Tooks like tsfresh and others can be used to systematically extract complex statistical features.

Expanded Window Length – Test a broader range of sequence window lengths for recurrent models

Have more relevant features in the dataset with more parameters for better performance.

References:

- Candanedo, L. M., Feldheim, V., & Deramaix, D. (2017). Data driven prediction models of energy use of appliances in a low-energy house. Energy and Buildings, 140, 81-97.
- UCI Machine Learning Repository — Appliances Energy Prediction Dataset.
- Optuna: A Next-generation Hyperparameter Optimization Framework (Akiba et al., 2019).
- TensorFlow / Keras documentation — LSTM, GRU, Conv1D layer references.

AI tools (Claude, Anthropic) were used throughout this project as a coding and technical reasoning partner, assisting with writing and debugging code, explaining the underlying mechanics of the models and techniques used, and helping structure this report. All modelling decisions, feature engineering choices, and their justifications were directed by the candidate based on their own analysis of the data; every result reported here was independently reviewed and validated, including identifying and correcting a data leakage issue in the feature selection pipeline. The candidate can explain and defend every technical decision made in this project.

-The End-